Here is a draft for your resume that perfectly matches the existing format and action-oriented tone of your other projects (like the Uber and Zomato analyses). It focuses heavily on the analytical value, the actionable insights, and the technical execution.

Resume Entry

Market Sentinel: AI-Powered Equity Analysis Dashboard Developed a full-stack financial intelligence platform utilizing React and Python (FastAPI) to deliver real-time NSE stock data and market insights. Integrated the Google Gemini API to analyze historical price momentum, news headlines, and market sentiment, engineering a proprietary 0-100 scoring algorithm that synthesizes complex financial signals into clear, actionable metrics for investors.

Common High-Level Interview Questions & Answers

When interviewers see a project like this on an analyst/data-focused resume, they are usually looking to understand your product thinking, your ability to handle data responsibly, and your problem-solving skills. Here are the most likely high-level questions and how to answer them:

1. "What inspired you to build Market Sentinel, and what problem does it solve?"

How to answer: "I built Market Sentinel because retail investors often suffer from information overload. When looking at a stock, you have to check historical charts, read through dozens of news articles, and try to gauge overall market sentiment—which is incredibly time-consuming. I wanted to build a tool that aggregates all of that scattered data. Market Sentinel solves this by pulling real-time price data and feeding it into an AI engine to instantly synthesize the trend, the news impact, and the sentiment into one easy-to-read dashboard."

2. "How can you trust the AI? How do you ensure the insights are reliable and not just hallucinations?"

How to answer: "That was actually one of my primary concerns during development. I knew I couldn't just let the AI invent a score out of thin air. To ensure reliability, I designed a hybrid system. The AI is strictly used for *qualitative* tasks—like summarizing news headlines and reading public sentiment. But the actual 'Sentinel Score' is calculated using a deterministic algorithm I wrote. It blends hard, mathematical price momentum (40% weight) with keyword-driven news impact (40%) and sentiment consistency (20%). This ensures the final output is grounded in real data rather than just an AI guess."

3. "Can you walk me through the architecture and how the data flows from start to finish?"

How to answer: "Sure! It starts at the frontend, which is a responsive dashboard built with React. When a user searches for an NSE ticker, the frontend makes an asynchronous call to my Python backend (built with FastAPI). The backend immediately reaches out to Yahoo Finance to pull the 90-day historical price data. It then packages that data into a strict prompt and sends it to the Google Gemini API to fetch live news and analyze the market sentiment. Once the AI returns its structured analysis, my backend runs it

through the custom scoring algorithm, packages the historical chart data and the AI summary together, and sends a single, clean JSON response back to the frontend to be displayed."

4. "If you had another month to work on this, what features would you add next?"

How to answer: "Right now, the platform analyzes one stock at a time on demand. If I had more time, I would build out a 'Watchlist' feature with a background cron job. This scheduler would automatically pull data for a user's saved stocks at the end of every trading day, recalculate the Sentinel Scores, and flag any drastic sentiment changes. This would shift the platform from being just a reactive research tool into a proactive portfolio monitoring system."

5. "What was your biggest takeaway or learning from building an end-to-end product like this?"

How to answer: "My biggest takeaway was learning how to elegantly handle third-party API limits and errors. When relying on external data sources like Yahoo Finance or Google Gemini, things will inevitably fail or get rate-limited. I learned how to build resilient systems—implementing proper try/except blocks and fallback responses. For example, if the AI engine goes down or hits a limit, my backend is designed to gracefully degrade. It skips the AI analysis but still successfully returns the historical stock charts, ensuring the user's experience is never completely broken."

Here is a comprehensive prep guide for your interview. It covers the high-level pitch as well as the technical deep-dives that interviewers love to dig into.

1. "Can you explain this project?" (The Elevator Pitch)

How to answer: "Market Sentinel is a full-stack financial intelligence application that provides real-time data and AI-driven insights for Indian stock market (NSE) equities. I built the frontend using **React and Vite** to create a highly responsive, glassmorphic dashboard. For the backend, I used **Python with FastAPI** because of its incredible speed and native support for asynchronous requests.

The core of the application fetches live and historical stock data directly from Yahoo Finance. It then feeds that data into the **Google Gemini API** to generate a contextual analysis of the stock—including trend summaries, news headline impacts, and market sentiment. Finally, I built a custom algorithm that combines the AI's qualitative analysis with quantitative price momentum data to calculate a proprietary **Sentinel Score**, giving users a clear, 0-100 gauge of a stock's current strength."

---

2. "Why did you choose FastAPI over Flask or Django?"

How to answer: "I chose FastAPI primarily for its performance and modern developer experience. Financial applications require fast data retrieval, and FastAPI is built from the ground up to be asynchronous, which is perfect for I/O bound tasks like making external API calls to Yahoo Finance and Google Gemini concurrently. Furthermore, FastAPI's automatic data validation using Pydantic ensures that the data contracts between my frontend and backend are strictly enforced, which prevented a lot of runtime bugs."

3. "What was the biggest technical challenge you faced, and how did you solve it?"

How to answer: "The biggest challenge was deploying the Python backend to Vercel's Serverless Functions. Vercel has a strict 250 MB uncompressed size limit for serverless deployments.

Initially, my backend relied on heavy data science libraries like pandas, numpy, and scipy through a wrapper library to fetch stock data. This caused the deployment to crash completely because the bundle size was nearly 500 MB. To solve this, I completely refactored the data-fetching layer. I stripped out the heavy dependencies and rewrote the service to use httpx to make direct, lightweight REST API calls to Yahoo Finance. This slashed the bundle size by over 80%, allowing the app to deploy perfectly on Vercel's free tier."

4. "How are you handling the AI integration, and what happens if the AI fails or gets rate-limited?"

How to answer: "I integrated the Google Gemini API to process the raw financial data and return structured JSON insights. Because I am using a free-tier API key that has strict rate limits (like 20 requests per day for certain models), I had to build a resilient fallback system.

First, the application attempts to use the primary model. If it hits a rate limit (429 RESOURCE_EXHAUSTED), the backend catches the exception and gracefully attempts a fallback model. If all AI models are exhausted or the API key is missing, the backend catches the ValueError and returns a default, neutral JSON response. This ensures that the frontend never crashes and the user can always still access the historical price charts and data, even if the AI engine is temporarily offline."

5. "Can you explain how the 'Sentinel Score' is calculated? Is it just the AI's opinion?"

How to answer: "No, it's actually a hybrid algorithm that prevents AI hallucinations from dictating the score. The AI only provides qualitative text summaries. The backend takes that text and uses a custom algorithmic scoring engine based on three weighted factors:

1. **Price Momentum (40%):** Pure math. It calculates the 90-day return and compares the moving average of the last 10 days against the previous 10 days to dynamically gauge momentum.

2. **Headline Impact (40%):** It scans the AI's news summary using a financial keyword algorithm, counting positive words (like *'growth', 'surge'*) vs negative words (like *'decline', 'risk'*) to calculate a net impact score.

3. **Sentiment Consistency (20%):** It extracts the raw sentiment labels from the AI's analysis and maps them to a numerical value.

By blending hard mathematical momentum with AI-driven contextual analysis, the Sentinel Score provides a much more grounded and reliable metric."

6. "How did you handle CORS and API routing between the frontend and backend?"

How to answer: "For production on Vercel, I configured a

vercel.json file to automatically rewrite all requests from the frontend aiming at /api/(.*) directly to the Python serverless function at

api/index.py.

For local development, I didn't want to hardcode localhost URLs in my React components, as that causes issues when pushing to production. Instead, I configured the Vite development server with a Proxy. Any request the frontend makes to /api locally is intercepted by Vite and forwarded to the FastAPI server running on port 8000. This allowed me to use clean, relative URLs (/api/analyze/TICKER) across the entire codebase that work perfectly in both local and production environments."

I'd be happy to explain that! It's actually a really interesting approach. The **Sentinel Score** is a hybrid score (from 0 to 100) that doesn't just rely on AI; it combines hard mathematical historical data with the AI's contextual analysis of the current market.

Here is exactly how the score is calculated under the hood. It uses a **weighted formula** based on three factors:

1. 📈 Price Trend & Momentum (40% Weight)

This part is pure math and doesn't rely on the AI at all. It looks at the 90-day historical chart:

- It calculates the total percentage return from 90 days ago to today.
- **Momentum Bonus:** It compares the average price of the *last 10 days* against the average price of the *previous 10 days* (days 11-20).

- Strong positive returns and upward momentum score higher towards 100.

2. 📰 Headline Impact / News (40% Weight)

This takes the qualitative news summary generated by Gemini and mathematically scores it using a financial keyword algorithm:

- **Positive Keywords:** It scans the AI's summary for words like *"growth", "surge", "upgrade", "expansion", "profit", "bullish"*, etc.
- **Negative Keywords:** It scans for words like *"decline", "loss", "downgrade", "risk", "debt", "penalty", "slowdown"*.
- It subtracts the negative keyword count from the positive count to generate a net score.

3. 🧠 Sentiment Consistency (20% Weight)

Gemini writes a summary on the overall public market sentiment (what retail/institutional investors are feeling), and the formula assigns a raw score based on the specific language the AI uses:

- *"Strong pattern"* or *"Consistent Bullish"* = **85**
- *"Bullish"* = **70**
- *"Positive"* = **65**
- *"Mixed"* = **50**
- *"Bearish"* = **30**
- *"Consistent/Strong Bearish"* = **15**

The Final Result The system multiplies those three scores by their weights (40% + 40% + 20%), adds them together, and caps the result between 0 and 100.

That final number is then mapped to the labels you see on the dashboard:

- **81 - 100:** Strong Catalyst
- **61 - 80:** Bullish Lean
- **41 - 60:** Neutral
- **21 - 40:** Cautionary
- **0 - 20:** Distressed